Решающие деревья.

Пицик Харитон

Клуб ML

Определение решающего дерева

`Classification and Regression Trees (CART)` состоят из:

- Корневой узел;

Определение решающего дерева

`Classification and Regression Trees (CART)` состоят из:

- Корневой узел;
- Ветви;

Определение решающего дерева

`Classification and Regression Trees (CART)` состоят из:

- Корневой узел;
- Ветви;
- Решающие (внутренние) узлы, листовые (терминальные) узлы.

Определение решающего дерева

`Classification and Regression Trees (CART)` состоят из:

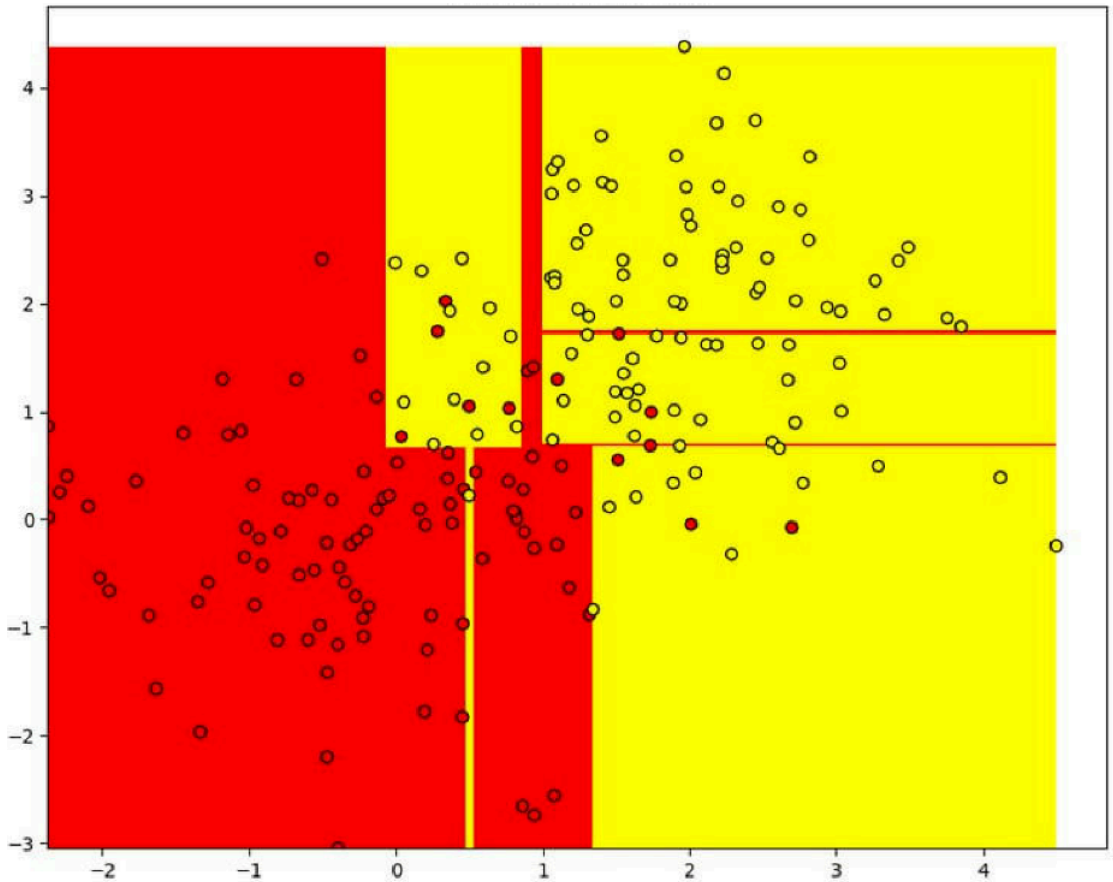
- Корневой узел;
- Ветви;
- Решающие (внутренние) узлы, листовые (терминальные) узлы.

По факту, `CART` очень похож на бинарное дерево (коим и является в большинсте случаев).

Определение решающего дерева

Сразу будем полагать, что R_m — выборка в рассматриваемом узле, R_l, R_r — объекты, которые попадут в левую или в правую ветвь соответственно.

Также будем считать, что в любой момент для узла можно посчитать c_m, на случай если мы посчитаем нужной объявить его листовым.


4
3
2
1
0
−1
−2
−3
−2
−1
0
1
2
3
4

Проблема градиентного спуска

Проблема градиентного спуска

Можем ли мы найти такое решающее дерево, которое:

Проблема градиентного спуска

Можем ли мы найти такое решающее дерево, которое:

- Идеально разделит выборку;

Проблема градиентного спуска

Можем ли мы найти такое решающее дерево, которое:

- Идеально разделит выборку;
- Не переобучено?

Проблема градиентного спуска

Можем ли мы найти такое решающее дерево, которое:

- Идеально разделит выборку;
- Не переобучено?

Как Вы могли догадаться, это NP полная задача.

Проблема градиентного спуска

Помимо этого, не очень похоже на то, что мы можем тут как-то применить градиентный спуск.

Соответственно, надо придумать какой-то жадный алгоритм:

Проблема градиентного спуска

Помимо этого, не очень похоже на то, что мы можем тут как-то применить градиентный спуск.

Соответственно, надо придумать какой-то жадный алгоритм:

```
если выполнен критерий остановы:
```

Проблема градиентного спуска

Помимо этого, не очень похоже на то, что мы можем тут как-то применить градиентный спуск.

Соответственно, надо придумать какой-то жадный алгоритм:

```
если выполнен критерий остановы:
  находим cm
```

Проблема градиентного спуска

Помимо этого, не очень похоже на то, что мы можем тут как-то применить градиентный спуск.

Соответственно, надо придумать какой-то жадный алгоритм:

```
если выполнен критерий остановы:
  находим cm
  объявляем вершину листовой
```

Проблема градиентного спуска

Помимо этого, не очень похоже на то, что мы можем тут как-то применить градиентный спуск.

Соответственно, надо придумать какой-то жадный алгоритм:

```
если выполнен критерий остановы:
  находим cm
  объявляем вершину листовой
  выход
конец
```

Что за критерий остановы?

Критерий остановы (в контексте нашей лекции) — условие, при соблюдении которого мы прекращаем строить дерево, предполагая, что оптимальное уже построенно.

Что за критерий остановы?

Критерий остановы (в контексте нашей лекции) — условие, при соблюдении которого мы прекращаем строить дерево, предполагая, что оптимальное уже построенно.

Например:

- Если достигли максимальной глубины (указывается параметром);

Что за критерий остановы?

Критерий остановы (в контексте нашей лекции) — условие, при соблюдении которого мы прекращаем строить дерево, предполагая, что оптимальное уже построенно.

Например:

- Если достигли максимальной глубины (указывается параметром);
- Если количество объектов в узле меньше некоторого порога.

Оптимальные параметры предиката

Будем их искать, используя формулу

$$j_*, t_* = \arg\max_{j,t} Q(R_m, j, t)$$